GPU CUDA Project

Cartographer - Fast correlative scan matcher

심인욱 (Professor)
실시간 데이터 해석 실습 자료
이예빈
Robotics and Computer Vision (RCV) lab.
Inha University, Republic of Korea

00 Overview

GPU Parallelization of a Scan matcher in Cartographer

✓ Goal

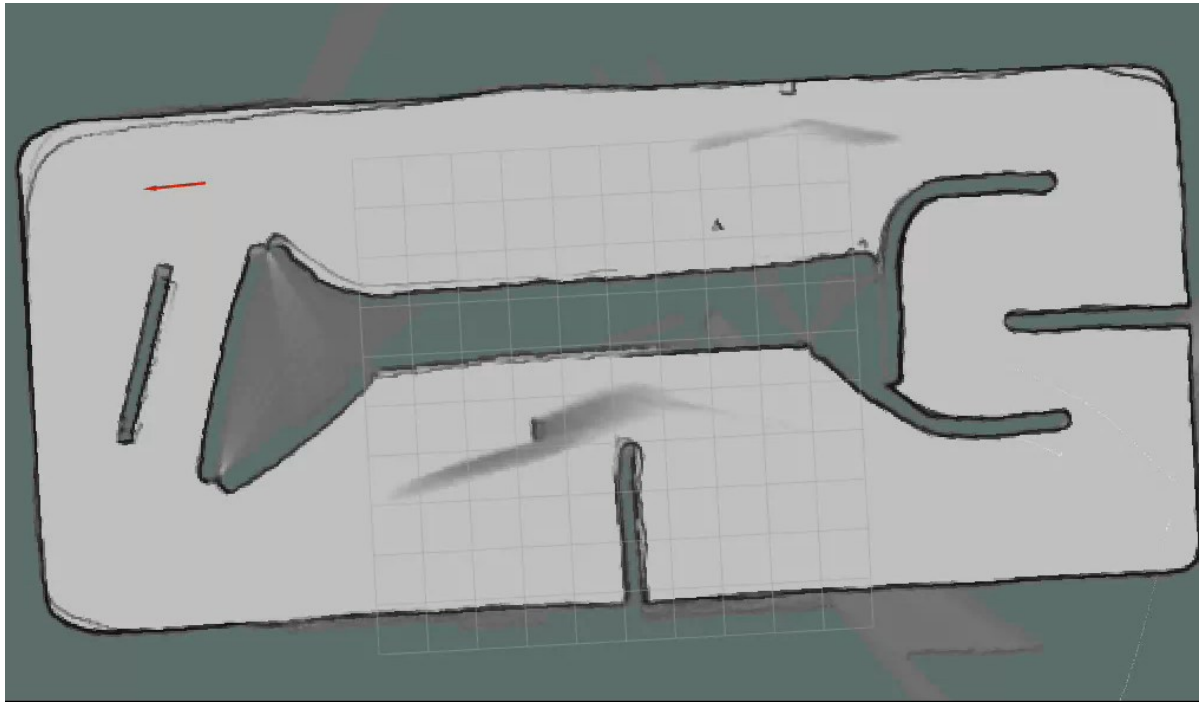
- Implement GPU parallelization using CUDA
- Compare CPU and GPU results and runtime

✓ Using Cartographer

- Use a simplified scan matching module from Cartographer
- Focus on one computation-heavy function : score_all()

✓ Cartographer

- 2D LiDAR SLAM frame work by Google
- Estimate robot poses by matching scan to map



Used Example in RCV-Formula

01 Cartographer

✓ Cartographer Pipeline

Local SLAM

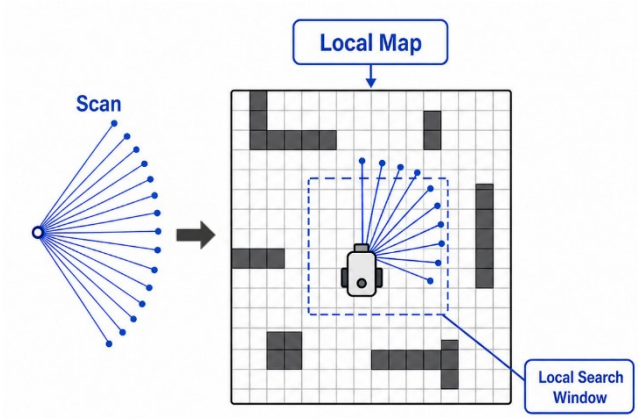
LaserScan

↓

Scan - Local map matching

↓

Pose Estimate



Global SLAM

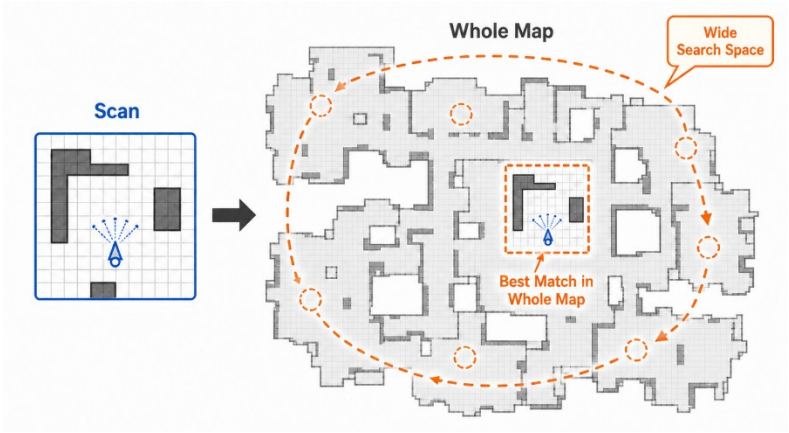
Local SLAM Result

↓

Scan-Global map matching

↓

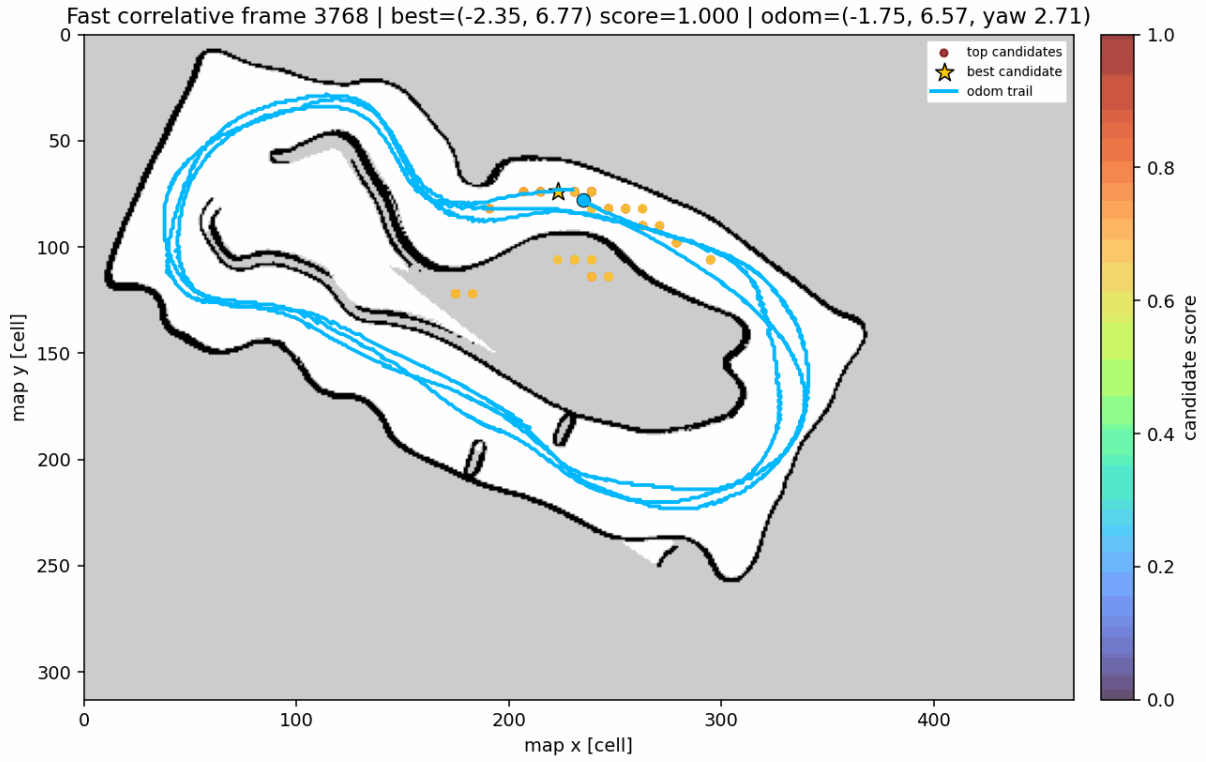
Pose Graph Optimization



01 Fast correlative scan matcher

✓ Fast correlative scan matcher

- Find the best alignment between a scan and map
- Search over multiple pose candidates (x, y, yaw)



Generate Pose Candidates

↓

Matching Score Calculation

↓

Best Match

01 Fast correlative scan matcher

✓ Code Structure

Generate Pose Candidates

- MakeScans() : Generate rotated scan hypothesis
- MakeBounds() : Compute search bounds for each scan hypothesis
- MakeLowCands() : Generate coarse X-Y candidates offsets

Matching Score Calculation

- Score() : Compute candidate scores and sort them
  - **score_all()**

    : Compute scan-to-map matching score for each candidate

Best Match

- Branch() : Refine candidates using branch-and-bound search
- ToOut() : Convert the selected candidate to output pose

02 Project: Target Function – score_all()

✓ score_all ()

- **Role**

  - Compute scan-to-map matching score for each candidate

  - Output : one score per candidate

- **Pseudo code**

```
for each candidate:
    for each scan point:
        x = px[j] + cx[i]
        y = py[j] + cy[i]
        sum += grid[y * w + x]
    score[i] = sum / normalization
```

- Simple repeated loop structure
- Computation size : candidates x scan points

02 File Structure

✓ Files for Assignment

- **Package Structure**

```
cartographer_parallel/
├── src/
│   ├── fast_correlative_node.cpp
│   ├── fast_matcher.cpp
│   └── score_all.cpp
├── include/cartographer_parallel/
│   ├── fast_matcher.h
│   └── assignment.h
├── launch/
│   ├── cartographer_parallel_with_bag.launch
│   └── fast_correlative.launch
├── maps/
│   ├── 0501.yaml
│   └── 0501.pgm
└── bags/
    └── scan.bag
```

- **Main File**

▪ **Fast_matcher.cpp**

- Main Fast correlative Scan matcher pipeline
- Calls Score() → score_all()

▪ **Score_all.cpp**

- Contains baseline : make_cand() and score_all()
- Main file to modify for GPU parallelization

▪ **cartographer_parallel_with_bag.launch**

- Main launch file for this assignment
- Runs the matcher with the provided map and rosbag

03 How to Build and run

- ## Initial Setting
  - ### Docker Setting

```
sudo docker start "student_00" && sudo docker exec -it "student_00" /bin/bash
```

  - ### File Setting in your container

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
cp -r /data/cartographer_parallel .
```

  - ### Build ros package

```
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

- ## Run
  - ### Run launch

```
roslaunch cartographer_parallel cartographer_parallel_with_bag.launch ns:="student_00"
```

Plz Make sure to type this.

03 How to Check Output

- **Check topic list**

```
rostopic list | grep "student_00"
```

- **Visualize in RViz**

```
Map
- Topic: /"student_00"/map

Odometry
- Topic: /"student_00"/fast_correlative_odom

PoseArray
- Topic: /"student_00"/fast_correlative_candidates

MarkerArray
- Topic: /"student_00"/fast_correlative_markers
```